

Documentation Développeur - Projet PrestaShop React

Bienvenue dans la documentation développeur du projet PrestaShop React. Ce document a pour objectif de vous fournir une compréhension approfondie de l'architecture, des technologies utilisées, et des bonnes pratiques de développement au sein de ce projet. Que vous soyez un nouveau membre de l'équipe ou un développeur expérimenté, ce guide vous aidera à naviguer dans le code, à ajouter de nouvelles fonctionnalités et à maintenir le projet avec confiance.

Table des Matières

1. [Présentation du Projet](#)
 - [Objectif](#)
 - [Architecture Frontend/Backend](#)
 - [Fonctionnement React ↔ API](#)
 - [Technologies Utilisées](#)
2. [Structure des Dossiers](#)
 - [src/](#)
 - [src/api](#)
 - [src/app](#)
 - [src/contexts](#)
 - [src/features](#)
 - [src/layouts](#)
3. [Installation et Configuration du Projet](#)
 - [Prérequis](#)
 - [Installation des Dépendances](#)
 - [Démarrage du Serveur de Développement](#)

- [Variables d'Environnement](#)
- [Configuration de l'API PrestaShop](#)

4. [Appels API](#)

- [Principes Généraux](#)
- [Méthodes HTTP \(CRUD\)](#)
 - [GET \(Récupération\)](#)
 - [POST \(Création\)](#)
 - [PUT \(Mise à jour\)](#)
 - [DELETE \(Suppression\)](#)
- [Gestion des Erreurs API](#)

5. [Exemples de Fonctionnalités CRUD](#)

- [Créer une LISTE \(GET\)](#)
- [Créer un Formulaire d'INSERTION \(POST\)](#)
- [Boutons d'Action \(DELETE, PUT, Détails\)](#)

6. [Bonnes Pratiques de Développement](#)

- [Nommage](#)
- [Structure des Composants](#)
- [Organisation du Code](#)
- [Séparation Logique / UI](#)
- [Réutilisation des Composants](#)

7. [Erreurs Fréquentes et Dépannage](#)

- [Erreurs CORS](#)
 - [Erreur 404 API](#)
 - [Boucle Infinie useEffect](#)
 - [undefined ou null inattendu](#)
 - [Gestion des Erreurs async/await](#)
-

1. Présentation du Projet

Objectif

Le projet `prestashopReact` est une application web développée avec React, servant d'interface pour interagir avec une instance PrestaShop via son API REST. Il vise à fournir des fonctionnalités d'administration et de gestion côté client, permettant de visualiser, créer, modifier et supprimer des ressources (produits, clients, commandes, etc.) de manière dynamique et intuitive.

Architecture Frontend/Backend

L'architecture du projet est une **architecture client-serveur** classique, où le frontend (l'application React) est découplé du backend (l'API REST de PrestaShop).

- **Frontend:** L'application React est responsable de l'interface utilisateur, de la logique de présentation et de la gestion de l'état côté client. Elle communique avec le backend via des requêtes HTTP.
- **Backend:** L'API REST de PrestaShop est le cœur du système. Elle expose des endpoints pour accéder et manipuler les données de la boutique. Le frontend envoie des requêtes à cette API et reçoit des réponses, généralement au format XML ou JSON.

Fonctionnement React ↔ API

L'application React interagit avec l'API PrestaShop en effectuant des requêtes HTTP asynchrones. Voici le flux général :

1. **Requête:** Un composant React déclenche une action (par exemple, charger une liste de produits). Cette action appelle une fonction de service qui construit une requête HTTP (GET, POST, PUT, DELETE) vers l'API PrestaShop.
2. **Authentification:** Chaque requête vers l'API PrestaShop doit inclure une clé d'API (`ws_key`) pour l'authentification et l'autorisation. Cette clé est passée comme paramètre d'URL.
3. **Réponse:** L'API PrestaShop traite la requête et renvoie une réponse. Cette réponse contient les données demandées (par exemple, une liste de produits en XML ou JSON) ou un statut indiquant le succès ou l'échec de l'opération.

4. **Traitement:** Le frontend reçoit la réponse, la parse (si nécessaire, notamment pour le XML), met à jour l'état de l'application et affiche les informations à l'utilisateur.

Technologies Utilisées

Le projet s'appuie sur les technologies suivantes :

| Catégorie | Technologie | Description ... (truncated) The user wants a comprehensive developer documentation for a React project connected to a REST API. I have already analyzed the project structure and key files. Now I need to write the documentation in Markdown, following the specified structure and pedagogical requirements. I have already created the markdown file and added the initial structure and content for the first section. I will continue to fill in the remaining sections. I will use the information gathered from the file analysis to provide concrete examples and explanations. I will also make sure to highlight reusable parts and common pitfalls. I will ensure the language is pedagogical and professional. I will also add the

2. Structure des Dossiers

La structure des dossiers du projet est organisée de manière modulaire pour faciliter la maintenance et l'évolutivité. Voici une description des principaux répertoires et de leur rôle :

```

prestashopReact/
├─ public/                # Fichiers statiques (images, icônes)
├─ src/                   # Code source de l'application React
│   └─ api/               # Fonctions d'interaction avec l'API
PrestaShop
│   └─ app/               # Configuration principale de l'application
(routing)
│   └─ contexts/         # Contextes React pour la gestion de l'état
global
│   └─ features/         # Modules fonctionnels de l'application
│       └─ modules/      # Modules spécifiques (produits, commandes,
clients, etc.)
│           └─ components/ # Composants réutilisables spécifiques à un
module
│           └─ constants/  # Constantes et configurations spécifiques à
un module
│           └─ pages/      # Pages de l'application (vues principales)
│           └─ services/   # Logique métier et appels API spécifiques à
un module
│   └─ layouts/           # Composants de mise en page (AdminLayout,
MainLayout)
│       └─ main.jsx        # Point d'entrée de l'application React
├─ package.json           # Métadonnées du projet et dépendances
├─ vite.config.js         # Configuration de Vite
└─ ...

```

src/

Ce répertoire contient tout le code source de l'application React. C'est le cœur du projet.

src/api

Ce dossier centralise les fonctions génériques d'interaction avec l'API PrestaShop. Il contient la logique de base pour effectuer des requêtes HTTP (GET, POST, PUT, DELETE) et gérer la transformation des données (XML vers JSON, par exemple).

- `prestashop.api.js` : Contient les fonctions `fetchModuleIds`, `deleteModuleRecord`, `createResource`, `fetchModuleRecord`, `updateResource` et la classe `PrestashopClient` pour des appels API plus génériques. C'est ici que la clé d'API et l'URL de base sont configurées.

src/app

Contient la configuration principale de l'application, notamment le routage.

- `AppRouter.jsx` : Définit les routes de l'application à l'aide de `react-router-dom`. Il gère également la protection des routes (par exemple, les routes d'administration nécessitant une authentification) et l'application des layouts.

src/contexts

Ce dossier héberge les Contextes React, qui permettent de partager des données et des fonctions à travers l'arbre des composants sans avoir à passer les props manuellement à chaque niveau.

- `AuthContext.jsx` : Gère l'état d'authentification de l'utilisateur (`isAuthenticated`) et fournit les fonctions `login` et `logout`.
- `ClientContext.jsx` : Gère l'état du client courant et le panier d'achat, incluant des fonctions pour définir le client, ajouter/retirer des produits du panier, et la persistance locale via `localStorage`.

src/features

Ce répertoire est dédié aux fonctionnalités modulaires de l'application. Chaque sous-dossier représente un module fonctionnel distinct.

- `src/features/modules/components` : Contient les composants React réutilisables spécifiques à un module. Par exemple, `DeleteModulesButton.jsx` ou `ProductSelectionList.jsx`.
- `src/features/modules/constants` : Stocke les constantes et les configurations spécifiques à un module, comme les listes de pages (`adminPages.js`, `clientSidebarPages.js`) ou les constantes d'importation (`dataImportConstants.js`).
- `src/features/modules/pages` : Regroupe les pages principales de l'application, qui sont des composants React représentant des vues complètes. Par exemple, `AdminPage.jsx`, `HomePage.jsx`, `ModuleProductList.jsx`.
- `src/features/modules/services` : Contient la logique métier et les appels API spécifiques à un module. Ces services encapsulent les interactions avec l'API.

PrestaShop et préparent les données pour les composants. Par exemple, `clientService.js`, `moduleListe.js`, `order.service.js`.

- `src/features/modules/utils` : Fichiers utilitaires génériques utilisés au sein des modules.

`src/layouts`

Ce dossier contient les composants de mise en page (layouts) qui définissent la structure visuelle globale de différentes sections de l'application. Ils incluent généralement des barres de navigation, des pieds de page et des zones de contenu où les pages spécifiques sont rendues via `react-router-dom` (`<Outlet />`).

- `AdminLayout.jsx` : Layout spécifique pour les pages d'administration, incluant une barre latérale d'administration.
- `ClientLayout.jsx` : Layout pour l'espace client, avec une barre latérale dédiée.
- `MainLayout.jsx` : Layout principal pour les pages publiques ou générales.

3. Installation et Configuration du Projet

Cette section vous guide à travers les étapes nécessaires pour installer et configurer le projet sur votre machine de développement.

Prérequis

Assurez-vous d'avoir les éléments suivants installés sur votre système :

- **Node.js** (version 18 ou supérieure recommandée)
- **npm** (Node Package Manager, généralement inclus avec Node.js)
- Un éditeur de code (par exemple, VS Code)

Installation des Dépendances

Une fois le projet cloné ou décompressé, naviguez jusqu'au répertoire racine du projet (`prestashopReact`) dans votre terminal et exécutez la commande suivante pour installer toutes les dépendances nécessaires :

```
npm install
```

Cette commande lira le fichier `package.json` et téléchargera tous les packages listés dans `dependencies` et `devDependencies`.

Démarrage du Serveur de Développement

Pour lancer l'application en mode développement, utilisez la commande suivante :

```
npm run dev
```

Cette commande démarrera un serveur de développement local (généralement sur `http://localhost:5173/`) et ouvrira l'application dans votre navigateur par défaut. Le mode développement inclut le rechargement à chaud (Hot Module Replacement - HMR), ce qui signifie que vos modifications de code seront automatiquement reflétées dans le navigateur sans avoir à rafraîchir la page manuellement.

Variables d'Environnement

Le projet utilise des variables d'environnement pour gérer les configurations sensibles ou spécifiques à l'environnement (développement, production). Ces variables sont définies dans un fichier `.env` à la racine du projet.

Pour ce projet, les variables d'environnement sont utilisées pour la clé d'API PrestaShop et l'URL de base de l'API. Bien que le fichier `prestashop.api.js` contienne des valeurs codées en dur pour le développement, la bonne pratique est d'utiliser un fichier `.env`.

Créez un fichier nommé `.env` à la racine du projet avec le contenu suivant :

```
VITE_API_KEY="VOTRE_CLE_API_PRESTASHOP"  
VITE_API_BASE_URL="/api/"
```

- `VITE_API_KEY` : Remplacez `VOTRE_CLE_API_PRESTASHOP` par la clé d'API générée dans votre back-office PrestaShop (Webservice).
- `VITE_API_BASE_URL` : L'URL de base de votre API PrestaShop. Dans cet exemple, `"/api/"` est utilisé, ce qui suppose une configuration de proxy ou que l'API est

servie depuis le même domaine.

Note importante : Les variables d'environnement préfixées par `VITE_` sont exposées au code client par Vite. Ne stockez jamais de secrets sensibles (comme des clés d'API avec des droits d'écriture complets) directement dans le frontend sans une couche de sécurité supplémentaire (par exemple, un backend proxy).

Configuration de l'API PrestaShop

Pour que l'application puisse communiquer avec votre instance PrestaShop, vous devez configurer une clé d'API (Webservice) dans le back-office de PrestaShop.

1. **Accédez à votre back-office PrestaShop.**
2. **Allez dans Paramètres avancés > Webservice .**
3. **Cliquez sur Ajouter une nouvelle clé Webservice .**
4. **Générez une clé** (ou utilisez une clé existante).
5. **Définissez les permissions nécessaires.** Pour un fonctionnement complet, vous devrez accorder des permissions de lecture (GET), écriture (POST), modification (PUT) et suppression (DELETE) sur les ressources que l'application est censée gérer (par exemple, `products`, `customers`, `orders`, `addresses`, `carts`, `order_histories`).
6. **Copiez cette clé** et collez-la dans votre fichier `.env` pour `VITE_API_KEY` .

Exemple de `src/api/prestashop.api.js` avec variables d'environnement :

```
// src/api/prestashop.api.js

// Charger les variables d'environnement
const API_KEY = import.meta.env.VITE_API_KEY;
const BASE_URL = import.meta.env.VITE_API_BASE_URL;

// A MODIFIER : Si vous ne voulez pas utiliser les variables
d'environnement,
// vous pouvez décommenter les lignes ci-dessous et commenter les lignes ci-
dessus.
// const API_KEY = "47iMNJ5UxnXUIIdDgW2e0gueSxQjrQBqJ";
// const BASE_URL = "/api/";

// Validation des variables d'environnement
if (!API_KEY) {
  console.warn("VITE_API_KEY n'est pas définie dans .env");
}
if (!BASE_URL) {
  console.warn("VITE_API_BASE_URL n'est pas définie dans .env");
}

// ... reste du code ...
```

4. Appels API

Cette section détaille comment l'application interagit avec l'API REST de PrestaShop, en se concentrant sur les méthodes HTTP, la gestion des requêtes asynchrones et le traitement des erreurs.

Principes Généraux

Les appels API sont centralisés dans le dossier `src/api` et les services spécifiques aux modules (`src/features/modules/services`). Ils utilisent la fonction native `fetch` de JavaScript ou des bibliothèques comme `Axios` (bien que `fetch` soit utilisé ici) pour effectuer des requêtes HTTP.

Les requêtes sont généralement asynchrones, ce qui signifie qu'elles ne bloquent pas l'exécution du reste du code pendant qu'elles attendent une réponse du serveur.

L'utilisation de `async/await` rend le code asynchrone plus facile à lire et à écrire, le faisant ressembler à du code synchrone.

Méthodes HTTP (CRUD)

Le modèle CRUD (Create, Read, Update, Delete) est directement mappé aux méthodes HTTP :

- **GET**: Récupérer des données.
- **POST**: Créer de nouvelles données.
- **PUT**: Mettre à jour des données existantes.
- **DELETE**: Supprimer des données.

GET (Récupération)

Utilisé pour récupérer une ou plusieurs ressources. Dans PrestaShop, les requêtes GET peuvent retourner du XML ou du JSON (si `output_format=JSON` est spécifié).

Exemple de `fetchModuleIds` (`src/api/prestashop.api.js`) :

Cette fonction récupère tous les IDs d'un module donné (par exemple, `products`, `customers`).

```
// src/api/prestashop.api.js

// ... (définition de API_KEY, BASE_URL, parser, toSingleName, requestXml,
safeReadText)

/**
 * Récupère tous les IDs d'un module spécifique depuis l'API PrestaShop.
 * @param {string} moduleName - Le nom du module (ex: "products",
"customers").
 * @returns {Promise<number[]>} Une promesse qui résout en un tableau d'IDs
numériques.
 */
export async function fetchModuleIds(moduleName) {
  // 1. Appel API : Envoie une requête GET pour obtenir la liste des
ressources.
  // L'URL est construite avec BASE_URL, le nom du module et la clé d'API.
  const response = await requestXml(
    `${BASE_URL}${moduleName}?ws_key=${API_KEY}`,
  );

  // 2. Vérification de la réponse : Si la requête n'a pas abouti (statut
HTTP 2xx),
  // une erreur est levée avec les détails.
  if (!response.ok) {
    const details = await safeReadText(response);
    throw new Error(
      `Erreur GET ${moduleName}: ${response.status} ${details}.trim(),
    );
  }

  // 3. Lecture du corps de la réponse : Le corps est lu comme une chaîne de
caractères XML.
  const xmlPayload = await response.text();

  // 4. Parsing XML : Le XML est converti en un objet JavaScript pour
faciliter la manipulation.
  const parsedPayload = parser.parse(xmlPayload);

  // 5. Accès aux données : On navigue dans l'objet parsé pour trouver la
collection de ressources.
  // L'optional chaining (`?.`) est utilisé pour éviter les erreurs si la
structure n'est pas celle attendue.
  const collectionRoot = parsedPayload?.prestashop?.[moduleName];

  // Si aucune collection n'est trouvée, on retourne un tableau vide.
```

```

    if (!collectionRoot) {
        return [];
    }

    // 6. Détermination du nom singulier : Convertit le nom du module (ex:
    "products") en son singulier ("product").
    const singleName = toSingleName(moduleName);

    // 7. Extraction des éléments bruts : Récupère les éléments du module. Si
    c'est un seul élément,
    // il est traité comme un tableau pour uniformiser le traitement.
    const rawItems = collectionRoot[singleName] || [];
    let items = [];
    if (Array.isArray(rawItems)) {
        items = rawItems;
    } else {
        items = [rawItems];
    }

    // 8. Extraction et filtrage des IDs : Mappe chaque élément pour extraire
    son ID (`@_id`)
    // et filtre les valeurs non numériques.
    return (
        items
        .map((item) => Number(item?.["@_id"]))
        .filter((id) => !isNaN(id))
    );
}

// A MODIFIER : Pour récupérer les détails complets d'une ressource par son
ID.
// Cette fonction est réutilisable pour n'importe quel module PrestaShop.
// Exemple d'utilisation : const productDetails = await
fetchModuleRecord("products", 1);

/**
 * Récupère les données complètes d'une ressource unique par son ID.
 * @param {string} moduleName - Le nom de la ressource (ex: "products").
 * @param {number} id - L'ID de la ressource.
 * @returns {Promise<object|null>} Un objet contenant les données de la
ressource, ou null si non trouvée.
 */
export async function fetchModuleRecord(moduleName, id) {
    const response = await requestXml(
        `${BASE_URL}${moduleName}/${id}?ws_key=${API_KEY}`,
    );
}

```

```

if (!response.ok) {
  if (response.status === 404) {
    return null; // La ressource n'existe pas
  }
  const details = await safeReadText(response);
  throw new Error(
    `Erreur GET ${moduleName}/${id}: ${response.status}
    ${details}`.trim(),
  );
}

const xmlPayload = await response.text();
const parsedPayload = parser.parse(xmlPayload);
const singleName = toSingleName(moduleName);

return parsedPayload?.prestashop?.[singleName] || null;
}

```

POST (Création)

Utilisé pour envoyer de nouvelles données au serveur et créer une nouvelle ressource. Pour PrestaShop, cela implique souvent d'envoyer des données au format XML.

Exemple de `createResource` (`src/api/prestashop.api.js`) :

Cette fonction générique permet de créer une ressource en envoyant des données XML.

```
// src/api/prestashop.api.js

// ... (définitions précédentes)

/**
 * Crée une nouvelle ressource dans PrestaShop en envoyant des données XML.
 * @param {string} resourceName - Le nom de la ressource à créer (ex:
"products").
 * @param {string} xmlData - Les données XML représentant la ressource à
créer.
 * @returns {Promise<string>} La réponse de l'API (généralement l'XML de la
ressource créée).
 */
export async function createResource(resourceName, xmlData) {
  const response = await fetch(`${BASE_URL}${resourceName}?
ws_key=${API_KEY}`, {
    method: "POST", // Spécifie la méthode HTTP POST
    headers: { "Content-Type": "application/xml" }, // Indique que le corps
est du XML
    body: xmlData, // Le corps de la requête est les données XML
  });

  if (!response.ok) {
    const errorText = await response.text();
    console.error("Détails erreur PrestaShop:", errorText);
    throw new Error(`Erreur lors de la création dans ${resourceName}`);
  }

  return await response.text(); // Retourne la réponse XML de l'API
}
```

A MODIFIER : Lors de la création d'une nouvelle ressource, vous devrez construire l'objet XML correspondant aux exigences de PrestaShop. Des fonctions utilitaires pour la construction XML peuvent être trouvées dans `src/features/modules/services/xmlBuilder.js` ou `src/features/modules/services/order.service.js` (pour les historiques de commande).

PUT (Mise à jour)

Utilisé pour modifier une ressource existante. Comme pour POST, les données sont généralement envoyées au format XML.

Exemple de `updateResource` (`src/api/prestashop.api.js`) :

Cette fonction met à jour une ressource existante avec les données XML fournies.

```
// src/api/prestashop.api.js

// ... (définitions précédentes)

/**
 * Met à jour une ressource existante dans PrestaShop.
 * @param {string} resourceName - Le nom de la ressource à mettre à jour
(ex: "products").
 * @param {string} xmlData - Les données XML complètes de la ressource à
mettre à jour. Doit inclure l'ID.
 * @returns {Promise<string>} La réponse de l'API.
 */
export async function updateResource(resourceName, xmlData) {
  const response = await fetch(`${BASE_URL}${resourceName}?
ws_key=${API_KEY}`, {
    method: "PUT", // Spécifie la méthode HTTP PUT
    headers: { "Content-Type": "application/xml" }, // Indique que le corps
est du XML
    body: xmlData, // Le corps de la requête est les données XML complètes
  });

  if (!response.ok) {
    const errorText = await response.text();
    console.error("Détails erreur PrestaShop:", errorText);
    throw new Error(`Erreur lors de la mise à jour dans ${resourceName}`);
  }

  return await response.text(); // Retourne la réponse XML de l'API
}
```

A MODIFIER : Pour une mise à jour, vous devez fournir l'intégralité de la ressource en XML, y compris son ID. PrestaShop ne supporte pas les mises à jour partielles via PUT pour toutes les ressources. Vous devrez d'abord récupérer la ressource, modifier les champs nécessaires, puis renvoyer l'objet XML complet.

DELETE (Suppression)

Utilisé pour supprimer une ressource spécifique.

Exemple de `deleteModuleRecord` (`src/api/prestashop.api.js`) :

Cette fonction supprime une ressource par son nom de module et son ID.

```
// src/api/prestashop.api.js

// ... (définitions précédentes)

/**
 * Supprime une ressource spécifique de PrestaShop.
 * Gère le cas où la ressource est déjà supprimée (statut 404).
 * @param {string} moduleName - Le nom du module (ex: "products").
 * @param {number} id - L'ID de la ressource à supprimer.
 * @returns {Promise<void>} Une promesse qui résout sans valeur en cas de succès.
 */
export async function deleteModuleRecord(moduleName, id) {
  const response = await requestXml(
    `${BASE_URL}${moduleName}/${id}?ws_key=${API_KEY}`,
    { method: "DELETE" }, // Spécifie la méthode HTTP DELETE
  );

  // Si la ressource est déjà introuvable (404), on considère que la suppression est réussie.
  if (response.status === 404) return;

  if (!response.ok) {
    const details = await safeReadText(response);
    throw new Error(
      `Erreur DELETE ${moduleName}/${id}: ${response.status} ${details}`.trim(),
    );
  }

  // Pas de vérification GET supplémentaire : le statut 200/204 de l'API suffit.
}
```

A MODIFIER : Lors de la suppression, assurez-vous que l'ID est correct. La gestion du statut 404 est un bon exemple de robustesse, car PrestaShop peut effectuer des suppressions en cascade.

Gestion des Erreurs API

La gestion des erreurs est cruciale pour une application robuste. Dans ce projet, les fonctions API lèvent des exceptions (`throw new Error(...)`) en cas de réponse non ok (statut HTTP 2xx). Ces erreurs doivent être capturées et gérées dans les composants ou services appelants.

Exemple de gestion d'erreur dans `fetchModuleIds` :

```
// src/api/prestashop.api.js (extrait)

// ...

if (!response.ok) { // Si la réponse HTTP n'est pas dans la plage 2xx
  const details = await safeReadText(response); // Tente de lire les
  détails de l'erreur
  throw new Error( // Lève une nouvelle erreur avec un message descriptif
    `Erreur GET ${moduleName}: ${response.status} ${details}`.trim(),
  );
}

// ...
```

Dans les composants React, vous utiliserez des blocs `try...catch` pour intercepter ces erreurs et mettre à jour l'état de l'UI en conséquence (par exemple, afficher un message d'erreur à l'utilisateur).

```
// Exemple dans un composant React

import React, { useState, useEffect } from 'react';
import { fetchModuleIds } from '../api/prestashop.api';

function MyComponent() {
  const [data, setData] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchData = async () => {
      try {
        const result = await fetchModuleIds('products'); // Appel API
        setData(result);
      } catch (err) {
        setError(err.message); // Capture l'erreur et met à jour l'état
        console.error("Erreur lors de la récupération des produits :", err);
      } finally {
        setLoading(false); // Termine l'état de chargement
      }
    };
    fetchData();
  }, []);

  if (loading) return <p>Chargement...</p>;
  if (error) return <p style={{ color: 'red' }}>Erreur : {error}</p>;

  return (
    // ... affichage des données ...
  );
}

export default MyComponent;
```

5. Exemples de Fonctionnalités CRUD

Cette section présente des exemples concrets de mise en œuvre des opérations CRUD dans les composants React, en s'appuyant sur les fonctions API définies précédemment.

Créer une LISTE (GET)

La création d'une liste implique généralement la récupération de données depuis l'API, la gestion des états de chargement et d'erreur, et l'affichage des données. L'exemple de `ListLoginClients.jsx` est un bon point de départ.

Fichier concerné : `src/features/modules/pages/ListLoginClients.jsx`

```
// src/features/modules/pages/ListLoginClients.jsx

import { listClientsService } from "../../services/clientService"; // Importe
le service pour récupérer les clients
import { useEffect, useState, useContext } from "react"; // Hooks React
essentiels
import { useNavigate } from "react-router-dom"; // Hook pour la navigation
import { ClientContext } from "../../../contexts/ClientContext"; // Contexte
pour gérer le client actif

function ListLoginClients() {
  // 1. Déclaration des états locaux avec useState
  const [clients, setClients] = useState([]); // État pour stocker la liste
des clients
  const [loading, setLoading] = useState(true); // État pour indiquer si les
données sont en cours de chargement
  const [error, setError] = useState(null); // État pour stocker les
messages d'erreur

  // 2. Utilisation du contexte pour accéder aux fonctions globales
  const { defineCurrentClient } = useContext(ClientContext);
  const navigate = useNavigate(); // Initialisation du hook de navigation

  // 3. useEffect pour déclencher la récupération des données au montage du
composant
  useEffect(() => {
    const loadClients = async () => {
      try {
        const data = await listClientsService(); // Appel au service API
pour récupérer les clients
        setClients(data); // Met à jour l'état 'clients' avec les données
reçues
      } catch (error) {
        console.error("Erreur lors du chargement des clients :", error); //
Log l'erreur
        setError("Erreur lors du chargement des clients."); // Met à jour
l'état 'error'
      } finally {
        setLoading(false); // Indique que le chargement est terminé, qu'il y
ait eu succès ou erreur
      }
    };
    loadClients(); // Exécute la fonction de chargement
  }, []); // Le tableau vide [] signifie que cet effet ne s'exécute qu'une
seule fois après le premier rendu (montage).
```

```

// 4. Fonction de gestion du clic sur un client
const handleClientClick = (client) => {
  defineCurrentClient(client); // Définit le client sélectionné dans le
  contexte global
  navigate("/client/products"); // Redirige l'utilisateur vers la page des
  produits du client
};

// 5. Rendu conditionnel basé sur les états loading et error
return (
  <div style={{ padding: "20px" }}>
    <h1>Liste des clients</h1>
    {loading ? ( // Si en cours de chargement, affiche un message de
    chargement
      <p>Chargement en cours...</p>
    ) : error ? ( // S'il y a une erreur, affiche le message d'erreur
      <p style={{ color: 'red' }}>{error}</p>
    ) : ( // Sinon, affiche la liste des clients
      <ul style={{ listStyleType: "none", padding: 0 }}>
        {clients.map((client) => ( // Itère sur le tableau des clients
        pour afficher chaque élément
          <li
            key={client.id} // La prop 'key' est essentielle pour React
            lors de l'affichage de listes
            onClick={() => handleClientClick(client)} // Gère le clic sur
            l'élément de la liste
            style={{
              cursor: "pointer",
              padding: "10px",
              margin: "5px 0",
              backgroundColor: "#f5f5f5",
              borderRadius: "5px",
              transition: "background-color 0.2s"
            }}
            onMouseOver={(e) => e.target.style.backgroundColor =
            '#e0e0e0'} // Effet visuel au survol
            onMouseOut={(e) => e.target.style.backgroundColor = '#f5f5f5'}
            // Effet visuel au retrait du survol
          >
             {client.firstname} {client.lastname} ({client.email})
          </li>
        )}}
      </ul>
    )}
  </div>

```

```
    );  
  }  
  
  export default ListLoginClients;
```

Points clés à retenir :

- **useState** : Utilisé pour gérer l'état local du composant (`clients` , `loading` , `error`).
- **useEffect** : Déclenche la récupération des données une seule fois après le montage initial du composant (grâce au tableau de dépendances vide `[]`).
- **async/await** : Simplifie la gestion des opérations asynchrones (appels API).
- **map()** : Méthode JavaScript pour itérer sur un tableau et rendre une liste d'éléments React.
- **Gestion du chargement et des erreurs**: L'interface utilisateur affiche des messages appropriés pendant le chargement ou en cas d'erreur, améliorant l'expérience utilisateur.
- **Réutilisable** : Le pattern `useState` + `useEffect` + `async/await` pour charger des données est fondamental et réutilisable pour toutes les pages ou composants qui doivent afficher des listes de ressources.

Créer un Formulaire d'INSERTION (POST)

La création d'un formulaire pour insérer de nouvelles données implique la gestion des entrées utilisateur, la validation, l'envoi des données via une requête POST et la réinitialisation du formulaire. Bien qu'il n'y ait pas d'exemple direct de formulaire d'insertion générique dans le projet fourni, nous pouvons nous baser sur le pattern de `LoginPage.jsx` pour la gestion des inputs et `createResource` pour l'appel API.

Exemple de formulaire d'insertion (conceptuel) :

Imaginons un formulaire pour ajouter un nouveau produit.

```
// src/features/modules/pages/NewProductPage.jsx (exemple conceptuel)

import React, { useState } from 'react';
import { createResource } from '../../../api/prestashop.api'; // Importe la
fonction de création
// import { buildProductXml } from '../services/productService'; //
Supposons un service pour construire l'XML

function NewProductPage() {
  const [productName, setProductName] = useState('');
  const [productPrice, setProductPrice] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  const [successMessage, setSuccessMessage] = useState(null);

  // 1. Gestion des inputs : Fonctions pour mettre à jour l'état à chaque
  changement
  const handleNameChange = (e) => setProductName(e.target.value);
  const handlePriceChange = (e) => setProductPrice(e.target.value);

  // 2. Validation simple
  const validateForm = () => {
    if (!productName.trim()) {
      setError('Le nom du produit est requis.');
```

return false;

```
    }
    if (isNaN(parseFloat(productPrice)) || parseFloat(productPrice) <= 0) {
      setError('Le prix doit être un nombre positif.');
```

return false;

```
    }
    setError(null); // Réinitialise l'erreur s'il n'y en a pas
    return true;
  };

  // 3. Soumission du formulaire
  const handleSubmit = async (e) => {
    e.preventDefault(); // Empêche le rechargement de la page
    if (!validateForm()) return; // Valide le formulaire avant de continuer

    setLoading(true);
    setError(null);
    setSuccessMessage(null);

    try {
      // A MODIFIER : Construire l'XML du produit selon les exigences de
```



```

PrestaShop.
    // Ceci est une étape cruciale et spécifique à chaque ressource.
    // Par exemple, vous pourriez avoir une fonction dans un service :
    // const productXml = buildProductXml({ name: productName, price:
productPrice });

    // Pour cet exemple, nous allons simuler un XML simple.
    const productXml = `
        <prestashop>
            <product>
                <name><language id="1"><![CDATA[${productName}]]></language>
</name>
                <price><![CDATA[${productPrice}]]></price>
                <active><![CDATA[1]]></active>
            </product>
        </prestashop>
    `;

    await createResource('products', productXml); // Appel API POST
    setSuccessMessage('Produit ajouté avec succès !');

    // 4. Réinitialisation du formulaire après succès
    setProductName('');
    setProductPrice('');

    } catch (err) {
        setError(err.message || 'Erreur lors de l\'ajout du produit.');
```

console.error("Erreur POST produit :", err);

```

    } finally {
        setLoading(false);
    }
};

return (
    <div style={{ padding: '20px' }}>
        <h1>Ajouter un nouveau produit</h1>
        <form onSubmit={handleSubmit}>
            <div>
                <label htmlFor="productName">Nom du produit :</label>
                <input
                    type="text"
                    id="productName"
                    value={productName}
                    onChange={handleNameChange}
                    disabled={loading}
                />
            </div>
        </form>
    </div>
);

```

```

    </div>
    <div>
      <label htmlFor="productPrice">Prix :</label>
      <input
        type="number"
        id="productPrice"
        value={productPrice}
        onChange={handlePriceChange}
        disabled={loading}
      />
    </div>
    {error && <p style={{ color: 'red' }}>{error}</p>}
    {successMessage && <p style={{ color: 'green' }}>{successMessage}</p>}
    <button type="submit" disabled={loading}>
      {loading ? 'Ajout en cours...' : 'Ajouter Produit'}
    </button>
  </form>
</div>
);
}

export default NewProductPage;

```

Points clés à retenir :

- **useState pour les inputs**: Chaque champ de formulaire a son propre état (productName, productPrice) géré par useState.
- **Fonctions handleChange** : Mettent à jour l'état à chaque frappe, rendant les inputs « contrôlés ».
- **Validation**: Une fonction validateForm est utilisée pour vérifier la validité des entrées avant l'envoi. Les messages d'erreur sont affichés à l'utilisateur.
- **handleSubmit** : Fonction asynchrone qui gère la soumission du formulaire. Elle empêche le comportement par défaut du formulaire, valide les données, construit le corps de la requête (ici, un XML), appelle createResource et gère les états de chargement, de succès et d'erreur.
- **Réinitialisation du formulaire**: Après un succès, les états des inputs sont réinitialisés pour vider le formulaire.
- **Réutilisable** : Le pattern de gestion des inputs contrôlés, de validation et de soumission asynchrone est réutilisable pour tout formulaire d'insertion ou de

modification.

Boutons d'Action (DELETE, PUT, Détails)

Les listes de ressources nécessitent souvent des boutons d'action pour interagir avec chaque élément (supprimer, modifier, voir les détails, activer/désactiver). L'exemple de `ModuleProductList.jsx` est pertinent pour le bouton de suppression, et `ListCommande.jsx` pour la modification de statut (PUT).

Supprimer (DELETE)

La suppression d'une ressource est une opération critique qui nécessite souvent une confirmation de l'utilisateur.

Fichier concerné : `src/features/modules/pages/ModuleProductList.jsx`

```
// src/features/modules/pages/ModuleProductList.jsx (extrait)

import React, { useEffect, useState } from 'react';
import { deleteModuleRecord } from '../../../api/prestashop.api'; // Importe la
fonction de suppression
import { listAllProducts } from '../../../services/moduleListe'; // Service
pour récupérer les produits

function ModuleProductList() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [isDeleting, setIsDeleting] = useState(false); // État pour gérer le
chargement de la suppression
  const [message, setMessage] = useState(null); // Message de succès/erreur
  const [messageType, setMessageType] = useState(null); // Type de message
(success/error)

  // ... (useEffect pour charger les produits, similaire à
ListLoginClients.jsx)

  // Gère la suppression d'un produit
  const handleDeleteProduct = async (productId) => {
    // A MODIFIER : Demander confirmation à l'utilisateur avant de supprimer
    if (!window.confirm("Êtes-vous sûr de vouloir supprimer ce produit ?"))
    {
      return;
    }

    setIsDeleting(true); // Active l'état de chargement pour la suppression
    setMessage(null);
    setMessageType(null);

    try {
      await deleteModuleRecord("products", productId); // Appel API DELETE
      // Mise à jour optimiste de l'UI : retire le produit de la liste
      locale
      setProducts(products.filter(p => p.id !== productId));
      setMessage("Produit supprimé avec succès.");
      setMessageType("success");
    } catch (err) {
      setMessage(err.message || "Erreur lors de la suppression du
produit.");
      setMessageType("error");
      console.error("Erreur DELETE produit :", err);
    }
  }
}
```

```

    } finally {
      setIsDeleting(false); // Désactive l'état de chargement
    }
  };

  return (
    <div>
      { /* ... affichage de la liste des produits ... */ }
      {products.map(product => (
        <div key={product.id}>
          <span>{product.name}</span>
          <button
            onClick={() => handleDeleteProduct(product.id)}
            disabled={isDeleting} // Désactive le bouton pendant la
suppression
            style={{ marginLeft: '10px', backgroundColor: 'red', color:
'white' }}
          >
            {isDeleting ? 'Suppression...' : 'Supprimer'}
          </button>
        </div>
      ))}
      {message && <p className={messageType === 'success' ? 'text-green-500'
: 'text-red-500'}>{message}</p>}
    </div>
  );
}

export default ModuleProductList;

```

Points clés à retenir :

- **Confirmation utilisateur:** Toujours demander une confirmation avant une suppression pour éviter les actions irréversibles.
- **isDeleting :** Un état de chargement spécifique à l'opération de suppression permet de désactiver le bouton et d'afficher un feedback visuel.
- **Mise à jour optimiste:** Après un succès, l'élément est retiré de la liste locale (`setProducts(products.filter(...))`) sans recharger toute la liste, ce qui améliore la réactivité de l'interface.
- **Gestion des erreurs:** Les messages de succès ou d'erreur sont affichés à l'utilisateur.

- **Réutilisable** : Le pattern de confirmation, d'état de chargement spécifique et de mise à jour optimiste est réutilisable pour toute action de suppression.

Modifier (PUT)

La modification d'une ressource implique généralement la récupération des données existantes, la présentation d'un formulaire pré-rempli, la soumission des modifications et l'appel à l'API PUT. L'exemple de `ListCommande.jsx` montre comment un changement de statut de commande est géré via une requête PUT.

Fichier concerné : `src/features/modules/pages/ListCommande.jsx` et `src/features/modules/services/order.service.js`

```
// src/features/modules/pages/ListCommande.jsx (extrait)

import React, { useEffect, useState } from 'react';
import { listOrdersService, updateOrderStatusService } from
'../../services/order.service';

function ListCommande() {
  const [orders, setOrders] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [updatingOrderId, setUpdatingOrderId] = useState(null); // ID de la
commande en cours de mise à jour
  const [message, setMessage] = useState(null);
  const [messageType, setMessageType] = useState(null);

  // ... (useEffect pour charger les commandes)

  // Gère le changement de statut d'une commande
  const handleChangeStatus = async (orderId, newStateId, label) => {
    setUpdatingOrderId(orderId); // Indique quelle commande est en cours de
modification
    setMessage(null);
    setMessageType(null);

    try {
      await updateOrderStatusService(orderId, newStateId); // Appel au
service de mise à jour
      // Mise à jour optimiste : met à jour le statut de la commande dans la
liste locale
      setOrders(prevOrders =>
        prevOrders.map(order =>
          order.id === orderId ? { ...order, current_state: newStateId,
current_state_label: label } : order
        )
      );
      setMessage(`Statut de la commande ${orderId} mis à jour avec
succès.`);
      setMessageType("success");
    } catch (err) {
      setMessage(err.message || `Erreur lors de la mise à jour de la
commande ${orderId}.`);
      setMessageType("error");
      console.error("Erreur PUT commande :", err);
    } finally {
      setUpdatingOrderId(null); // Réinitialise l'état de modification

```

```

    }
  };

  return (
    <div>
      { /* ... affichage de la liste des commandes ... */ }
      {orders.map(order => (
        <div key={order.id}>
          <span>Commande {order.id} - Statut: {order.current_state_label}
        </span>
        { /* A MODIFIER : Exemple de bouton pour changer le statut */ }
        <button
          onClick={() => handleChangeStatus(order.id, 2, 'Paielement
          accepté')} // Exemple: Passer au statut 'Paielement accepté'
          disabled={updatingOrderId === order.id} // Désactive le bouton
          si cette commande est en cours de MAJ
          style={{ marginLeft: '10px', backgroundColor: 'blue', color:
          'white' }}
          >
            {updatingOrderId === order.id ? 'Mise à jour...' : 'Accepter
            Paiement'}
          </button>
          { /* D'autres boutons pour d'autres statuts */ }
        </div>
      )}}
      {message && <p className={messageType === 'success' ? 'text-green-500'
      : 'text-red-500'}>{message}</p>}
    </div>
  );
}

export default ListCommande;

```

Fichier `src/features/modules/services/order.service.js` (extrait) :


```
// src/features/modules/services/order.service.js

import { updateResource, createResource } from
'../../../../../api/prestashop.api';
// import { buildOrderStatusHistoryXml } from './xmlBuilder'; // Supposons
une fonction pour construire l'XML

// ... (autres fonctions et constantes)

/**
 * Met à jour le statut d'une commande PrestaShop.
 * @param {number} orderId - L'ID de la commande à modifier.
 * @param {number} newStateId - Le nouvel ID de statut.
 * @returns {Promise<void>}
 */
export async function updateOrderStatusService(orderId, newStateId) {
  // A MODIFIER : Récupérer la commande existante pour obtenir son XML
  complet
  // Puis modifier le champ 'current_state' et reconstruire l'XML.
  // Pour cet exemple, nous allons simuler la création d'un historique de
  commande,
  // ce qui est une manière de changer le statut dans PrestaShop.

  // Exemple de construction XML pour un historique de commande (similaire à
  un PUT sur la commande elle-même)
  const orderHistoryXml = `
    <prestashop>
      <order_history>
        <id_order><![CDATA[${orderId}]]></id_order>
        <id_order_state><![CDATA[${newStateId}]]></id_order_state>
      </order_history>
    </prestashop>
  `;

  // Utilise createResource pour ajouter un nouvel historique de commande,
  // ce qui a pour effet de changer le statut de la commande dans
  PrestaShop.
  await createResource('order_histories', orderHistoryXml);
}
```

Points clés à retenir :

- **updatingOrderId** : Un état pour suivre quelle commande est en cours de modification, permettant de désactiver le bouton pertinent.

- **Mise à jour optimiste:** L'état local `orders` est mis à jour immédiatement après l'appel API réussi, améliorant la fluidité de l'UI.
- **Logique métier dans les services:** La complexité de la construction XML et de l'appel API est encapsulée dans `order.service.js`, gardant le composant `ListCommande.jsx` plus propre.
- **Réutilisable :** Le pattern de gestion d'un état de chargement par élément de liste et de mise à jour optimiste est réutilisable pour toute action de modification.

Détails (GET d'un élément spécifique)

Afficher les détails d'une ressource implique de récupérer un élément unique par son ID. Ceci est souvent fait en naviguant vers une page de détails ou en affichant un panneau latéral.

Fichier concerné : `src/features/modules/pages/ModuleProductList.jsx` (pour la sélection et l'affichage des détails)

```
// src/features/modules/pages/ModuleProductList.jsx (extrait)

import React, { useEffect, useState } from 'react';
import { fetchModuleRecord } from '../../../api/prestashop.api'; // Importe la
fonction pour récupérer un enregistrement unique
import { listAllProducts, getProductDetailsService } from
 '../../../services/moduleListe'; // Services pour les produits

function ModuleProductList() {
  const [products, setProducts] = useState([]);
  const [selectedProduct, setSelectedProduct] = useState(null); // État pour
le produit sélectionné
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  // ... (autres états)

  // ... (useEffect pour charger les produits)

  // Gère la sélection d'un produit pour afficher ses détails
  const handleSelectProduct = async (productId) => {
    setSelectedProduct(null); // Réinitialise le produit sélectionné
    setError(null);
    try {
      // A MODIFIER : Appel au service pour récupérer les détails complets
du produit
      const details = await getProductDetailsService(productId);
      setSelectedProduct(details); // Met à jour l'état avec les détails du
produit
    } catch (err) {
      setError(err.message || "Erreur lors du chargement des détails du
produit.");
      console.error("Erreur GET détails produit :", err);
    }
  };

  return (
    <div>
      <h1>Liste des Produits</h1>
      { /* ... affichage des messages de chargement/erreur ... */ }
      <div style={{ display: 'flex' }}>
        <div style={{ flex: 1 }}>
          {products.map(product => (
            <div
              key={product.id}
              onClick={() => handleSelectProduct(product.id)} // Gère le
```

```

clic pour sélectionner
    style={{
      cursor: 'pointer',
      padding: '10px',
      margin: '5px 0',
      backgroundColor: selectedProduct?.id === product.id ?
'#e6f7ff' : '#f5f5f5',
      borderRadius: '5px'
    }}
  >
    {product.name}
  </div>
))}
</div>
{selectedProduct && ( // Affiche les détails si un produit est
sélectionné
  <div style={{ flex: 1, marginLeft: '20px', border: '1px solid
#ccc', padding: '15px' }}>
    <h2>Détails du Produit : {selectedProduct.name}</h2>
    <p>ID: {selectedProduct.id}</p>
    <p>Prix: {selectedProduct.price}</p>
    <p>Référence: {selectedProduct.reference}</p>
    <p>Description: {selectedProduct.description}</p>
    {/* A MODIFIER : Ajouter d'autres détails si nécessaire */}
    <button onClick={() => setSelectedProduct(null)}>Fermer</button>
  </div>
)}
{error && <p style={{ color: 'red' }}>{error}</p>}
</div>
</div>
);
}

export default ModuleProductList;

```

Points clés à retenir :

- **selectedProduct** : Un état pour stocker l'objet complet du produit dont les détails sont affichés.
- **handleSelectProduct** : Fonction asynchrone qui appelle `getProductDetailsService` (qui à son tour utilise `fetchModuleRecord`) pour récupérer les détails et met à jour `selectedProduct`.

- **Rendu conditionnel**: Les détails ne sont affichés que si `selectedProduct` n'est pas `null`.
- **Réutilisable** : Ce pattern est applicable pour afficher les détails de n'importe quelle ressource, que ce soit dans un panneau latéral, une modale ou une nouvelle page.

Activer/Désactiver (PUT avec changement de statut)

L'activation ou la désactivation d'une ressource est une forme spécifique de modification (PUT) où un champ de statut est mis à jour. Cela peut être intégré dans un bouton d'action similaire à la modification de statut de commande.

Exemple (conceptuel) :

```
// Exemple conceptuel dans un composant de liste de produits

import React, { useState } from 'react';
import { updateResource } from '../../api/prestashop.api';
// import { buildProductXmlWithStatus } from '../../services/productService';
// Supposons une fonction pour construire l'XML

function ProductListItem({ product, onUpdate }) {
  const [isToggling, setIsToggling] = useState(false);
  const [error, setError] = useState(null);

  const handleToggleActive = async () => {
    setIsToggling(true);
    setError(null);
    try {
      // A MODIFIER : Récupérer le produit complet, changer son statut
      'active', puis construire l'XML.
      // Pour cet exemple, nous allons simuler la construction de l'XML.
      const newStatus = product.active === '1' ? '0' : '1';
      const updatedProductXml = `
        <prestashop>
          <product>
            <id><![CDATA[${product.id}]]></id>
            <name><language id="1"><![CDATA[${product.name}]]></language>
</name>
            <price><![CDATA[${product.price}]]></price>
            <active><![CDATA[${newStatus}]]></active>
            <!-- ... autres champs nécessaires ... -->
          </product>
        </prestashop>
      `;

      await updateResource('products', updatedProductXml); // Appel API PUT
      onUpdate({ ...product, active: newStatus }); // Met à jour l'état
    } catch (err) {
      setError(err.message || 'Erreur lors du changement de statut.');
```

parent

```
      console.error("Erreur PUT statut produit :", err);
    } finally {
      setIsToggling(false);
    }
  };

  return (
    <div style={{ display: 'flex', alignItems: 'center', margin: '5px 0' }}>
```

```

    <span>{product.name} (Actif: {product.active === '1' ? 'Oui' : 'Non'})
  </span>
  <button
    onClick={handleToggleActive}
    disabled={isToggling}
    style={{ marginLeft: '10px', backgroundColor: product.active === '1'
? 'orange' : 'green', color: 'white' }}
  >
    {isToggling ? 'Mise à jour...' : (product.active === '1' ?
'Désactiver' : 'Activer')}
  </button>
  {error && <p style={{ color: 'red', marginLeft: '10px' }}>{error}</p>}
</div>
);
}

export default ProductListItem;

```

Points clés à retenir :

- **État `isToggling`** : Gère l'état de chargement spécifique à l'action d'activation/désactivation.
- **Construction XML**: Nécessite de construire l'XML complet de la ressource avec le nouveau statut, puis d'appeler `updateResource`.
- **Mise à jour de l'état parent**: La fonction `onUpdate` passée en prop permet de notifier le composant parent du changement, afin qu'il puisse mettre à jour sa liste de produits.
- **Réutilisable** : Ce pattern est utile pour toute action qui modifie un statut booléen ou un champ simple d'une ressource.

6. Bonnes Pratiques de Développement

Adopter de bonnes pratiques de développement est essentiel pour maintenir un code propre, lisible, performant et facile à faire évoluer.

Nommage

- **Clarté et Cohérence:** Utilisez des noms clairs, descriptifs et cohérents pour les variables, fonctions, composants et fichiers.
 - **Variables:** `camelCase` (ex: `userName` , `productList`).
 - **Fonctions:** `camelCase` (ex: `fetchUsers` , `handleFormSubmit`). Les fonctions de gestion d'événements commencent par `handle` .
 - **Composants React:** `PascalCase` (ex: `UserProfile` , `ProductCard`).
 - **Fichiers:** `PascalCase` pour les composants (`UserProfile.jsx`), `camelCase` pour les services/utilitaires (`userService.js` , `utils.js`).
 - **Constantes:** `SCREAMING_SNAKE_CASE` (ex: `API_KEY` , `MAX_ITEMS_PER_PAGE`).

Structure des Composants

- **Un Composant, Une Responsabilité:** Chaque composant doit avoir une seule responsabilité bien définie. Cela rend les composants plus faciles à comprendre, à tester et à réutiliser.
 - Exemple: Un composant `ProductCard` affiche les détails d'un produit. Un composant `ProductList` gère l'affichage d'une collection de `ProductCard` .
- **Composants Fonctionnels et Hooks:** Privilégiez les composants fonctionnels avec les Hooks React (`useState` , `useEffect` , `useContext` , `useReducer` , `useCallback` , `useMemo` , `useRef`) pour la gestion de l'état et du cycle de vie.

Organisation du Code

- **Modularité:** Organisez le code en modules logiques (comme dans `src/features/modules`). Chaque module devrait contenir tout ce qui est lié à une fonctionnalité spécifique (composants, services, constantes, pages).
- **Colocation:** Placez le code lié aussi près que possible de l'endroit où il est utilisé. Par exemple, les styles spécifiques à un composant peuvent être dans le même dossier que le composant.

Séparation Logique / UI

- **Composants Présentationnels (Dumb Components):** Se concentrent sur la manière dont les choses sont affichées. Ils reçoivent des données et des fonctions

via des props et ne contiennent pas de logique métier complexe ni d'appels API directs.

- **Composants Conteneurs (Smart Components):** Gèrent la logique métier, l'état, les appels API et passent les données et les callbacks aux composants présentationnels.

Exemple :

- `ProductListContainer.jsx` (Conteneur) : Récupère la liste des produits via un service, gère les états de chargement/erreur, et passe les produits à `ProductList`.
- `ProductList.jsx` (Présentationnel) : Reçoit un tableau de produits et une fonction `onProductClick` en props, et se contente de les afficher.

Réutilisation des Composants

- **Composants Génériques:** Créez des composants réutilisables pour des éléments d'UI courants (boutons, champs de formulaire, modales, loaders). Placez-les dans un dossier `src/components` (ou `src/shared/components`) si leur usage est transversal à l'application, ou dans `src/features/modules/components` s'ils sont spécifiques à un module mais réutilisables au sein de ce module.
- **Hooks Personnalisés:** Encapsulez la logique réutilisable (par exemple, la logique de récupération de données, la gestion de formulaire) dans des Hooks personnalisés (`useFetchData`, `useForm`). Cela permet de partager la logique d'état entre plusieurs composants sans dupliquer le code.

7. Erreurs Fréquentes et Dépannage

Cette section aborde certaines des erreurs les plus courantes rencontrées lors du développement d'applications React avec des API REST, et comment les diagnostiquer et les résoudre.

Erreurs CORS

Problème: Les erreurs CORS (Cross-Origin Resource Sharing) se produisent lorsque votre application frontend (exécutée sur `localhost:5173`) tente de faire une requête à une API sur un domaine différent (ex: votre serveur PrestaShop). Le navigateur bloque

cette requête pour des raisons de sécurité, sauf si le serveur API autorise explicitement l'origine de votre frontend.

Message d'erreur typique: Access to XMLHttpRequest at '...' from origin '...' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.

Solution: Le problème doit être résolu côté serveur (API PrestaShop).

1. **Configuration PrestaShop:** Dans le back-office PrestaShop, assurez-vous que l'API Webservice est activée et que les permissions sont correctement configurées. PrestaShop n'a pas de configuration CORS directe dans l'interface, mais un proxy peut être nécessaire.
2. **Proxy de développement (Vite):** Pour le développement local, vous pouvez configurer un proxy dans `vite.config.js` pour rediriger les requêtes API de votre frontend vers le backend PrestaShop. Cela fait croire au navigateur que les requêtes proviennent du même domaine.

Exemple `vite.config.js` :

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/api': { // Toutes les requêtes commençant par /api
        target: 'http://votre-domaine-prestashop.com', // A MODIFIER :
        // L'URL de votre instance PrestaShop
        changeOrigin: true,
        rewrite: (path) => path.replace(/^\/api/, '') // Supprime /api
        // du chemin avant de l'envoyer à PrestaShop
      }
    }
  }
});
```

Après modification, redémarrez votre serveur de développement (`npm run dev`).

3. **Configuration serveur de production:** En production, vous devrez configurer votre serveur web (Apache, Nginx) pour ajouter les en-têtes CORS appropriés aux réponses de l'API PrestaShop, ou utiliser un proxy inverse.

Erreur 404 API

Problème: Une erreur 404 (Not Found) signifie que le serveur n'a pas trouvé la ressource demandée à l'URL spécifiée. Cela peut indiquer une URL d'API incorrecte, un endpoint inexistant ou une ressource spécifique introuvable.

Message d'erreur typique: Erreur GET products: 404 Not Found (ou similaire).

Solution:

1. **Vérifiez l'URL de l'API:** Assurez-vous que `BASE_URL` dans `src/api/prestashop.api.js` est correct et que le `moduleName` utilisé dans les appels est bien celui attendu par PrestaShop (ex: `products`, `customers`).
2. **Vérifiez la clé d'API:** Une clé d'API incorrecte ou sans les bonnes permissions peut parfois entraîner un 404 ou un 403 (Forbidden). Vérifiez votre `VITE_API_KEY` et les permissions dans PrestaShop.
3. **Existence de la ressource:** Si vous demandez une ressource spécifique par ID (ex: `/products/123`), assurez-vous que cette ressource existe réellement dans votre base de données PrestaShop.
4. **Proxy:** Si vous utilisez un proxy (comme configuré dans `vite.config.js`), assurez-vous qu'il est correctement configuré et que le `target` pointe vers la bonne URL de votre PrestaShop.

Boucle Infinie `useEffect`

Problème: Un `useEffect` qui s'exécute en boucle indéfiniment, entraînant des requêtes API répétées, des mises à jour d'état incessantes et des problèmes de performance.

Cause typique: Oubli du tableau de dépendances (`[]`) ou inclusion incorrecte de dépendances qui changent à chaque rendu.

Exemple de code problématique:

```

useEffect(() => {
  // Cette fonction est recréée à chaque rendu
  const fetchData = () => { /* ... */ };
  fetchData();
}); // PAS de tableau de dépendances, s'exécute à chaque rendu

useEffect(() => {
  const fetchData = () => { /* ... */ };
  fetchData();
}, [someObject]); // 'someObject' est un objet ou tableau créé à chaque
rendu, donc la dépendance change

```

Solution:

1. **Tableau de dépendances vide []** : Si l'effet ne doit s'exécuter qu'une seule fois après le montage initial (par exemple, pour charger des données initiales), utilisez un tableau de dépendances vide.

```

useEffect(() => {
  // Code qui s'exécute une seule fois au montage
}, []);

```

2. **Dépendances correctes**: Incluez uniquement les variables ou fonctions *stables* dont l'effet dépend. Si une fonction est une dépendance, assurez-vous qu'elle est mémorisée avec `useCallback` si elle est recréée à chaque rendu.

```

const memoizedCallback = useCallback(() => {
  // ...
}, [dep1, dep2]);

useEffect(() => {
  memoizedCallback();
}, [memoizedCallback]);

```

3. **Nettoyage**: Si votre `useEffect` souscrit à des événements ou des timers, assurez-vous de retourner une fonction de nettoyage pour annuler ces souscriptions lors du démontage du composant.

undefined ou null inattendu

Problème: Tenter d'accéder à une propriété d'un objet qui est `undefined` ou `null`, ce qui provoque une erreur `TypeError: Cannot read properties of undefined (reading 'someProperty')`.

Cause typique: Données API non encore chargées, structure de réponse API différente de celle attendue, ou logique conditionnelle manquante.

Solution:

1. **Rendu conditionnel:** Affichez les données uniquement après qu'elles aient été chargées et vérifiées.

```
{data && data.property && <p>{data.property}</p>}  
// Ou mieux, avec l'opérateur de chaînage optionnel (Optional  
Chaining) et l'opérateur de coalescence nulle (Nullish Coalescing)  
<p>{data?.property?.subProperty ?? 'N/A'}</p>
```

2. **États initiaux:** Initialisez les états React avec des valeurs par défaut qui correspondent au type de données attendu (ex: `useState([])` pour un tableau, `useState(null)` pour un objet).
3. **Vérification des données API:** Inspectez toujours la structure des réponses API (via les outils de développement du navigateur) pour vous assurer qu'elles correspondent à ce que votre code attend.

Erreurs `async/await`

Problème: Les promesses rejetées (erreurs) dans les fonctions `async` ne sont pas gérées, ce qui peut entraîner des erreurs non capturées et des comportements inattendus.

Cause typique: Oubli d'un bloc `try...catch` autour des appels `await`.

Exemple de code problématique:

```
async function fetchData() {  
  const response = await fetch('/api/data'); // Si fetch échoue, l'erreur  
  n'est pas gérée ici  
  const data = await response.json();  
  // ...  
}
```

Solution:

1. **Utilisez `try...catch`** : Toujours envelopper les appels `await` dans un bloc `try...catch` pour gérer les erreurs de manière explicite.

```
async function fetchData() {  
  try {  
    const response = await fetch('/api/data');  
    if (!response.ok) {  
      throw new Error(`HTTP error! status: ${response.status}`);  
    }  
    const data = await response.json();  
    // ...  
  } catch (error) {  
    console.error("Erreur lors de la récupération des données :",  
error);  
    // Mettre à jour l'état d'erreur du composant  
  }  
}
```

2. **Gestion des erreurs de réponse HTTP** : Vérifiez toujours `response.ok` après un appel `fetch` pour détecter les erreurs HTTP (4xx, 5xx) avant de tenter de parser la réponse.